



通信工程学院

嵌入式系统编程 实验指导书

2016 年 4 月

目 录

概述.....	1
实验一 文件操作.....	2
实验二 进程操作.....	6
实验三 多线程与内存管理.....	11
实验四 项目管理与综合实验.....	15
附录 A Linux 系统应用程序编译.....	21
附录 B 程序的调试.....	24
附录 C 项目管理与自动化工具.....	27
附录 D 版本控制与源代码管理.....	31

概述

根据“嵌入式系统编程”课程的教学大纲，课程中设置了 16 学时的实验。其主要目的是通过实验的配合，帮助掌握 Linux 操作系统下使用 C 语言开发应用程序的知识，理解操作系统应用程序接口的基本工作方式。

通过理论课的学习和课程内的实验，学会 Linux 系统中的编辑、编译、链接、装入过程。了解源代码的版本管理概念和常见的版本管理工具。掌握程序开发的过程和步骤，重要的技术模式。为在嵌入式电子系统中，基于 Linux 系统的嵌入式系统应用程序开发打下基础，掌握 Linux 系统中应用程序的开发流程和基本技术。

实验包括了针对文件和目录的操作、有关进程控制和进程通信的操作，多线程应用与内存管理，以及更具综合性的实验和项目管理手段。

实验环境的选择，优先考虑主流的发行版为基础。可以选用 Ubuntu/Debian 或者 Fedora/Redhat 发行套件，部署实验环境。课外的自学，可以选用 Knoppix Live CD，Ubuntu Live CD 或者使用 VirtualBox、Virtual PC 虚拟机安装方案，便于在个人计算机上尝试和学习。

实现项目中，部分示范代码的内容，可从实验室所提供的服务器下载获得，具体的下载地址由实验室提供。

1. 实验项目列表

序号	实验项目	实验学时
1	文件操作	4 学时
2	进程操作	4 学时
3	多线程与内存管理	4 学时
4	项目管理与综合实验	4 学时

2. 实验报告要求

实验过程中，应做好实验笔记。根据实验笔记撰写实验报告。实验报告由实验目的、实验过程、操作步骤、系统响应、错误修正等内容构成。应在实验报告中写明实验结果所说明的问题、实验所获得的体会。

3. 实验成绩评定

实验成绩根据实验准备、实验过程、实验报告等几个部分的完成情况综合评定。

4. 分项实验指导

实验一 文件操作

实验项目：文件操作

实验目的：掌握 Linux 系统中 C 编写应用程序的方法。学习 Linux 的文件系统的程序接口，掌握与文件和目录有关的操作。

实验内容：应用程序编辑、编译、链接、运行、调试的过程；针对目录和文件的编程操作。

实验用时：4 学时

首先确认你自己已经完全理解并掌握 Linux 操作系统中应用程序的编辑、编译、链接、运行、和调试的过程，如果有疑问，请转到附录参考相应的内容，做必要的准备。

1. 文件操作

针对文件的操作可以分为针对文件内容的操作，和针对文件属性等信息的操作。后者在我们的实验安排中，归入目录操作之内。因为这是目录项和文件索引结点所涉及到的内容。

对文件内容的操作，主要有打开和关闭文件、文件的读写、文件指针的概念、缓冲区对于文件读写性能的影响等。可以选择自己感兴趣的内容设计实验。也可参考以下内容进行实验。

研究 who 命令的工作机制，编写自己的 who 命令。

who 命令根据文件 /var/run/utmp 内部所记录的数据，报告当前系统中的已登录用户的相关信息。

尝试在命令行中使用 who 显示登录用户的信息。

尝试使用 cat 命令显示文件 /var/run/utmp 内容。

提示：如果上面的命令输出的乱码导致仿真终端的窗口显示混乱，可以输入 reset 并按回车键，然后用 Ctrl-L 组合键，清理窗口。

程序读取文件 /var/run/utmp 中存放的记录，按照系统预先定义的数据结构，用易读的方式显示登录用户的有关信息。

下面是关键的数据结构：

```
struct utmp {
    short    ut_type;           /* Type of record */
    pid_t    ut_pid;           /* PID of login process */
    char     ut_line[UT_LINESIZE]; /* Device name of tty - "/dev/" */
    char     ut_id[4];          /* Terminal name suffix,
                                or inittab(5) ID */
    char     ut_user[UT_NAMESIZE]; /* Username */
    char     ut_host[UT_HOSTSIZE]; /* Hostname for remote login, or
                                kernel version for run-level
                                messages */
    struct    exit_status ut_exit; /* Exit status of a process
                                marked as DEAD_PROCESS; not
                                used by Linux init(8) */
    /* The ut_session and ut_tv fields must be the same size when
    compiled 32- and 64-bit. This allows data files and shared
    memory to be shared between 32- and 64-bit applications. */
};
```

```

#if __WORDSIZE == 64 && defined __WORDSIZE_COMPAT32
    int32_t ut_session;          /* Session ID (getsid(2)),
                                   used for windowing */

    struct {
        int32_t tv_sec;          /* Seconds */
        int32_t tv_usec;        /* Microseconds */
    } ut_tv;                    /* Time entry was made */
#else
    long ut_session;             /* Session ID */
    struct timeval ut_tv;        /* Time entry was made */
#endif

int32_t ut_addr_v6[4];          /* Internet address of remote
                                   host; IPv4 address uses
                                   just ut_addr_v6[0] */
char __unused[20];              /* Reserved for future use */
};

```

这样的每一个记录，包含一个登录用户的信息。

注意个别用户为系统内部所使用，并非真正的登录用户。这可以通过 `ut_type` 成员是否等于 `USER_PROCESS` 加以区别。

仿照系统提供的 `cat`、`cp` 命令，自己动手编写简化了的 `cat`、`cp` 程序。

`cat` 命令显示文件的内容。注意如何判断文件的结束。

`cp` 命令需要使用创建文件的系统调用，并且要同时读取一个文件和写入一个文件。

按照你自己的需要，扩展你自己的 `cat` 命令和 `cp` 命令。例如：允许 `cat` 命令使用多个文件作为参数；当 `cat` 命令不带任何参数时，自动由标准输入读取数据；`cp` 命令检查目标文件是否存在，当目标文件存在时，给用户选择覆盖、追加、放弃等选择的机会。

2. 目录操作

尝试使用 `tree` 命令显示某个目录及其下属子目录。有时候，系统中可能并没有安装 `tree` 实用工具。这时，可以使用 `ls -R <路径名>` 命令递归地显示指定目录以及其下属的子目录内容。也可以是使用 `find <路径名>` 命令显示目录。

在观察过系统工具所显示的目录树之后，自己动手编写一个自己的 `tree` 命令，去显示指定目录的目录结构。

程序中的要点是：

用 `DIR *dp` 定义子目录流指针，使用 `opendir(dir)` 打开目录，获得指向目录流的指针。凭借这个指针，反复使用 `entry = readdir(dp)` 操作得到该目录中的每一个目录项。

每一个目录项，拥有自己特定的数据

```

struct dirent {
    ino_t      d_ino;          /* inode number */
    off_t      d_off;          /* not an offset; see NOTES */
    unsigned short d_reclen;    /* length of this record */
    unsigned char d_type;       /* type of file; not supported
                                   by all filesystem types */
    char       d_name[256];     /* filename */
};

```

```
};
```

通过其成员 `d_name` 就可以得到文件或下级子目录的名字。但是它究竟是目录还是文件，需要去查看它的文件类型。Linux 系统中并不依据上面的成员函数 `d_type` 作为判据，而是需要进一步读取文件信息，通过文件的模式加以判断。

使用 `lstat(entry->d_name, &statbuf)` 这样的操作，将文件的属性读入到 `statbuf` 指针所指向的结构体中。这个结构体，就是用来保存文件信息的结构体，如下：

```
struct stat {
    dev_t      st_dev;      /* ID of device containing file */
    ino_t      st_ino;      /* inode number */
    mode_t     st_mode;     /* protection */
    nlink_t    st_nlink;    /* number of hard links */
    uid_t      st_uid;      /* user ID of owner */
    gid_t      st_gid;      /* group ID of owner */
    dev_t      st_rdev;     /* device ID (if special file) */
    off_t      st_size;     /* total size, in bytes */
    blksize_t  st_blksize;  /* blocksize for filesystem I/O */
    blkcnt_t   st_blocks;   /* number of 512B blocks allocated */
    time_t     st_atime;    /* time of last access */
    time_t     st_mtime;    /* time of last modification */
    time_t     st_ctime;    /* time of last status change */
};
```

其中的成员 `st_mode` 如果拥有 `S_IFDIR` 对应的位，那它对应的文件就是一个目录文件。最安全的方法是好似用预先定义的宏 `S_ISDIR(st_mode)` 去判断。

如果确认是下一级的子目录，那就可以使用递归调用，去现实下一级目录的内容，从而实现多级目录树的递归显示。特别要在注意的是，在整个递归过程中，一定要跳过两个特殊的文件，一个是代表本目录的“.”，另一个是代表上级目录的“..”，如果不注意这一点，程序就会进入死循环。

研究系统的 `ls` 命令工作机制，编写自己 `ls` 命令。

`ls` 命令的主要功能是显示目录内容以及文件属性。目录内容的读取和显示与上面 `tree` 命令的实现方法基本相同。文件属性的获取需要用到 `stat` 函数，例如：

```
struct stat info;

if ( stat(filename, &info) == -1 )      /* cannot stat */
    perror( filename );                  /* say why */
else                                     /* else show info */
    show_file_info( filename, &info );
```

获得文件的相关信息之后，就可以根据前文提到的 `stat` 结构体中所定义的各成员函数，显示出文件的各项信息了。

有一些问题需要注意。首先文件名和文件的信息是分别存放在不同数据结构中的。文件名存放在目录项之内，文件的其他信息来自与文件的 `i` 节点信息，也就是 `stat` 结构体，所以要将文件名和文件信息都显示出来，需要将这两方面的信息综合在一起。

其次要注意，文件的属主和文件的组别信息，其原始的数据形式是系统内部的用户号和组号，这些要转换成用户名和组名显示出来才便于阅读。

还要注意文件的模式信息中，既包含了文件类型信息，又包含了文件的访问权限信息，需要细心地将模式值，转换成类似于“-rwxrwxrwx”这样的字符串显示，才符合一般的惯例。

为了帮助同学，针对以上内容，提供了部分程序的参考代码，具体的访问网址，请向实验指导教师咨询。

实验二 进程操作

实验项目：进程操作

实验目的：掌握进程概念，熟悉进程状态及其转换，学习进程控制相关的系统调用与函数。

实验内容：状态进程的获取；创建新进程，执行新进程，父子进程同步，进程的终止；进程间通信。

实验用时：4 学时

运行当中的系统，包含许多进程，每个进程可能在不同的时刻处于不同的状态，进程之间通过有机的合作，实现系统的功能。首先用 ps 命令观察系统中的进程。

Linux 系统中的 ps 命令有不同风格的选项形式。在 ps 命令的帮助手册中，专门有描述这个问题，分别是 UNIX 风格，BSD 风格，GNU/Linux 风格。通常可以使用 ps -ef 或者 ps aux 命令显示系统中所有的进程。

类似地，使用 pstree 命令可以显示出系统中的进程树。

注意观察进程的状态，以及进程状态的变化。

1. 进程控制

用户通过现有进程创建新的进程，使用 fork 调用。使用 exec 调用让进程去加载并执行新的代码。使用 wait 调用在父子进程之间同步。所有这些函数，都在系统调用库的头文件 unistd.h 之内定义。

下面的例子展示了父进程创建子进程，父子进程向同一个文件中写入内容，父进程在子进程终止后，才最后退出的例子：

```
#include <sys/types.h>           // 该头文件提供系统调用的标志
#include <sys/stat.h>             // 该头文件提供系统状态信息和相关函数
#include <sys/uio.h>              // 该头文件提供进程 I/O 操作的相关函数
#include <unistd.h>               // 标准函数库
#include <fcntl.h>                // 文件操作相关函数库
#include <string.h>               // 字符串操作函数库
#include <sys/wait.h>             // wait 调用相关函数库
#include <stdio.h>                // 标准输入与输出函数库
#include <stdlib.h>               // 常用工具函数库
int main()
{
    char buf[100] = {0};          // 定义缓冲区
    pid_t cld_pid;                // 定义该结构保存子进程的 PID
    int fd;
    int status;                   // 用于 wait 调用时的参数
    if ((fd = open("temp", O_CREAT | O_RDWR | O_TRUNC, 0664)) == -1)
    {
        perror("创建文件");
        exit(1);
    }
    strcpy(buf, "父进程数据");
    if ((cld_pid = fork()) == 0) { // 创建子进程，并判断自己是否是子进程
```



```

strcpy(buf, "子进程数据");
puts("子进程正在工作: ");
printf("子进程 PID 是%d\n", getpid());      // 输出子进程的 PID
printf("父进程 PID 是%d\n", getppid());      // 输出父进程的 PID
write(fd, buf, strlen(buf));
close(fd);
exit(0);
}
else {
    puts("父进程正在工作: ");
    printf("父进程 PID 是%d\n", getpid());    // 输出父进程的 PID
    printf("子进程 PID 是%d\n", cld_pid);     // 输出子进程的 PID
    write(fd, buf, strlen(buf));
    close(fd);
}
wait(&status);                             // 等待子进程结束
return 0;
}

```

实际上,当进程创建一个子进程时,通常是希望子进程执行和父进程不同的代码,完成与父进程不同的任务,所以在子进程中常常通过 `exec` 类的调用去执行另外的代码。这个过程,正好是 `shell` 在接受用户命令去运行其他程序时所做的工作。`shell` 在这个方面的工作可以用一个循环来概括:

- (1) `shell` 显示 `$` 提示符,等待用户输入命令;
- (2) 用户键入 `./hello` 并回车;
- (3) `shell` 创建一个子进程,在子进程内装入并运行 `hello` 程序;
- (4) `shell` 自身进入睡眠状态,等待子进程的结束;
- (5) `hello` 程序结束时, `shell` 被唤醒;
- (6) `shell` 再次显示 `$` 提示符,等待用户输入下一个命令。

现在,自己动手来编写一个自己的 `shell` 程序:

```

/** prompting shell */

#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include <signal.h>

#define MAXARGS 20          /* cmdline args */
#define ARGLEN 100         /* token length */

main()
{
    char *arglist[MAXARGS+1]; /* an array of ptrs */
    int numargs;              /* index into array */
    char argbuf[ARGLEN];      /* read stuff here */
    char *makestring();       /* malloc etc */

    numargs = 0;
    while ( numargs < MAXARGS )
    {

```

```

printf("Arg[%d]? ", numargs);
if ( fgets(argbuf, ARGLEN, stdin) && *argbuf != '\n' )
    arglist[numargs++] = makestring(argbuf);
else
{
    if ( numargs > 0 ){      /* any args? */
        arglist[numargs]=NULL; /* close list */
        execute( arglist ); /* do it */
        numargs = 0;      /* and reset */
    }
}
return 0;
}

execute( char *arglist[] )
/*
 * use fork and execvp and wait to do it
 */
{
    int pid, exitstatus;      /* of child */

    pid = fork();             /* make new process */
    switch( pid ){
        case -1:
            perror("fork failed");
            exit(1);
        case 0:
            execvp(arglist[0], arglist);      /* do it */
            perror("execvp failed");
            exit(1);
        default:
            while( wait(&exitstatus) != pid )
                ;
            printf("child exited with status %d,%d\n",
                exitstatus>>8, exitstatus&0377);
    }
}

char *makestring( char *buf )
/*
 * trim off newline and create storage for the string
 */
{
    char *cp;

    buf[strlen(buf)-1] = '\0';      /* trim newline */
    cp = malloc( strlen(buf)+1 );    /* get memory */
    if ( cp == NULL ){               /* or die */
        fprintf(stderr, "no memory\n");
        exit(1);
    }
    strcpy(cp, buf);      /* copy chars */
    return cp;            /* return ptr */
}

```

这个程序在执行时，和真正的 shell 有所不同，这里的程序需要你分别输入命令、选项、参数等，最后当你直接回车时，它就把你前面输入的命令、选项、参数合成到一起，开

始执行你指定的命令。

另一方面，实际的 shell 所拥有的功能，远比这里自己编写的 shell 丰富得多，比如文件名替换功能、输入输出重定向功能、脚本支持等等。这里所实现的仅仅是其中的程序运行功能。你可以以此为起点，仿照实际的 shell 逐步增加一些功能，看看能否写出更加接近真实 shell 的程序。

关于进程控制的其他功能，请自行设计实验。

2. 进程通信

进程之间的通信有多种方式，常用的方式有有名管道、无名管道、消息队列、共享内存段、内存映射文件等等。一些进程通信方式可能在其他课程中已经被介绍到。建议重点研究一下信号。注意区分 Linux 系统中的信号 (signal) 和信号量 (semaphore)。

信号是 Linux 系统中在进程之间，或者内核与进程之间的事件通知机制。系统使用信号通知进程发生了某种事件，该事件通常与进程的当前活动异步发生。

使用命令 `kill -l` 可以观察到系统支持的信号种类。

下面的代码展示了一个有问题的程序：

这段程序可以被成功编译，但是执行时会发生错误。因为里面试图想要读取一个它无权读取的内存地址，所以这段程序在执行时会发生所谓的“段错误” (segment fault)，这会让程序因为崩溃而退出。

```
/* an example that dereferences a null pointer */

int main(int argc, char **argv)
{
    int a; a = *(int *)0;    /* read address 0 */
}
```

如果在程序中捕获了段错误的信号，有可以对错误进行妥善的处理，从而可以让程序可以更加体面的退出。实际上也不是“面子”问题，一旦你拥有了更多的控制能力，就可以在程序退出之前收集一些程序在运行中的数据，为将来排查程序中的错误保留一些线索。也可能是调用另外的子程序做出某种补救性的处理。

```
/* signal example #1: trap an illegal address reference */

#include <stdio.h>    /* for printf */
#include <stdlib.h>    /* for exit */
#include <signal.h>    /* defines signals and the signal() function */

main(int argc, char **argv) {
    void catch(int);
    int a;

    signal(SIGSEGV, catch); /* catch SIGSEGV */
    a = *(int *)0;    /* read address 0 */
}

void
catch(int snum) {
    printf("got signal %d\n", snum);
    exit(0);
}
```

不同的事件对应不同的信号。下面的程序利用了定时信号（SIGALRM）让程序每隔一段时间，去执行一段程序。

```
#include<stdio.h>
#include<signal.h>
#include<unistd.h>

void sig_handler(int signo)
{
    printf("\nB-e-e-p! 3 sencond passed.\n\n");
    alarm(2);
}

int main(void)
{
    if (signal(SIGALRM, sig_handler) == SIG_ERR)
        printf("\ncan't catch SIGALRM\n");

    alarm(2);
    long i=0;
    while (1) {
        printf("I am doging hard work. i = %ld now. \n", i);
        sleep(1);
        i++;
    }
    return 0;
}
```

实验三 多线程与内存管理

实验项目：Linux 线程机制

实验目的：学习利用 Linux 的线程机制实现并发任务

实验内容：线程创建、终止、线程合作、线程通信、互斥锁。

实验用时：4 学时

1. 线程的基础知识

Linux 系统中的 pthread 线程库是用于 C 的一个标准 API。它让程序员可以创建新的并发流程。不论是多处理、多核、或者单一处理器的环境下，多个线程的并发，都有可能获得效率上的提升。和创建新的进程不同，因为不必分配新的内存空间，无需初始化新进程，使用线程的开销更少。

线程操作包括：线程的创建，终止，同步（join，block），调度，数据管理和进程交互。线程不负责记录它所创建的其他线程，也不管是谁创建了它自己。一个进程中的所有线程共享相同的地址空间。

同一进程中的线程，这些是相同的：

- 进程指令代码
- 绝大多数的数据
- 所打开的文件（描述符）
- 信号和信号句柄
- 当前工作目录
- 用户身份号和组身份号

下面这些，是每个线程各自不同的：

- 线程身份号
- 寄存器集合，堆栈指针
- 局部变量的堆栈，返回地址
- 信号的掩码值
- 优先级
- 线程返回值：errno

2. 线程操作的基本步骤

在 Linux 系统中使用 pthread 库，首先需要包含 pthread.h 头文件。然后通过四个步骤创建一个基本的线程程序：

1) 定义线程引用变量

变量类型 pthread_t 的是用来定义线程变量的。需要有一个 pthread_t 的变量指明要创建的每个线程。

2) 为线程指明一个执行入口

使用 pthread 中创建一个线程时，需要让它指向一个函数去执行。该函数必须符合这样的格式，带一个 void * 类型的参数，并且返回值也是 void * 类型。

3) 创建线程

使用创建线程的函数 `pthread_create` 创建线程。该函数需要四个参数：一个指向 `pthread_t` 变量指针，一个指向说明线程属性的变量指针（通常设为 `NULL` 即可），一个指向线程函数入口的指针。

4) 等待新的线程结束后重新加入主线程

当新创建的线程完成它的使命，我们需要它重新 `join` 进来，恢复正常。这一工作通过 `pthread_join` 函数完成。它接受两个参数：用 `pthread_t` 类型的变量指明是哪个线程，用一个指向返回值指针来接收返回值，如果不关心返回值，设置为 `NULL` 即可。

3. 简单的例子

下面是一个线程应用程序的简单例子。它定义两个整型变量，`x` 和 `y`，并创建另外一个线程。主线程当中的 `y` 递增，直到 100。与此同时，在新创建的另外一个线中，`x` 递增到 100。第二个线程结束后，回到主线程打印结果。请注意，新创建的线程是怎样地改变了变量 `x` 的值。

因为这段程序使用了 `pthread` 库，所以在编译时，需要添加 `-lpthread` 选项。以便让 `gcc` 了解，在链接时将库文件 `libthread.so` 包含在内。

```
gcc program.c -o program -lpthread
```

下面是参考代码：

```
#include <pthread.h>
#include <stdio.h>

/* this function is run by the second thread */
void *inc_x(void *x_void_ptr)
{
    /* increment x to 100 */
    int *x_ptr = (int *)x_void_ptr;
    while(++(*x_ptr) < 100);

    printf("x increment finished\n");

    /* the function must return something - NULL will do */
    return NULL;
}

int main()
{
    int x = 0, y = 0;

    /* show the initial values of x and y */
    printf("x: %d, y: %d\n", x, y);

    /* this variable is our reference to the second thread */
    pthread_t inc_x_thread;

    /* create a second thread which executes inc_x(&x) */
    if(pthread_create(&inc_x_thread, NULL, inc_x, &x)) {
```

```

fprintf(stderr, "Error creating thread\n");
return 1;

}
/* increment y to 100 in the first thread */
while(++y < 100);

printf("y increment finished\n");

/* wait for the second thread to finish */
if(pthread_join(inc_x_thread, NULL)) {

fprintf(stderr, "Error joining thread\n");
return 2;

}

/* show the results - x is now 100 thanks to the second thread */
printf("x: %d, y: %d\n", x, y);

return 0;

}

```

以上程序代码运行的结果如下：

```

x: 0, y:0
y increment finished
x increment finished
x: 100, y: 100

```

4. 实验延伸

在以上程序合适的地方，本别增加 printf 语句，各自输出 x, y 的值，为了控制速度，可以在程序中加上 sleep(2) 函数语句，获得 2 秒的延时。观察输出数字的变化规律和交替位置，体会线程之间并发。

5. 其他

线程之间的通信可以直接通过共享全局变量实现。但是对全局变量的访问，需要妥善协调。涉及多个线程之间合作，需要以下线程同步机制：

互斥锁：

```

pthread_mutex_lock
pthread_mutex_unlock
pthread_mutex_trylock

```

条件变量：

```

pthread_cond_init
pthread_cond_destroy
pthread_cond_wait
pthread_cond_timedwait
pthread_cond_signal
pthread_cond_broadcast

```

这些函数的使用，请参考联机手册（使用 man 命令可以查看联机手册），并设计自行

设计实验进行尝试。

实验四 项目管理与综合实验

实验项目：项目管理与综合实验

实验目的：通过一个综合性的实验，学习 Linux 系统中更加复杂的软件项目管理技术。

实验内容：建立用户的数据结构，学习内存的分配与管理，文件系统的访问，多进程的通信与合作。

实验用时：4 学时

1. 模拟实验环境

假设在一个温度测量系统中，安装在测点附近的嵌入式系统，包含有一个温度传感器，每隔几秒钟，温度传感器会侧得一个当前的温度值，通过嵌入式系统中的程序，该温度值被送到数据处理中心的计算机上。如图 1 所示：



图 1：温度测量系统示意图

为了在实验室内模拟这样的系统，我们用两台联网的计算机来模拟：使用鼠标模拟温度传感器，使用终端显示器模拟数据汇集点上的通信接口。如图 2 所示：



图 2：温度测量系统的实验模拟

Linux 系统中的鼠标，被抽象成一个设备，通常是：`/dev/input/mice`，访问这设备可以得到鼠标移动的数据。我们移动鼠标，用以模拟温度传感器，向嵌入式系统发送温度数据。计算机 A 获取这些用来代表温度（或者温度变化量）的数据，并将它们通过网络送给同一个网络上的计算机 B。

计算机 B 上的终端显示器上可以显示程序输出的文字，终端显示器也是 Linux 系统缺省输出的标准设备。在终端中显示计算机 B 所获得的“温度值”，代表整个“温度测量系统”的正常工作所产生的结果。

2. 基本要求与提示

实验的基本要求是用 C 语言编制程序，分别运行在计算机 A 和计算机 B 上，实现模拟的温度测量系统。在实验中，请综合利用此前学习过的所有知识，包括使用 `make` 进行多模块的编译，使用 `git` 进行版本控制与源代码管理。

鼠标运动数据的获得，可以通过读取本地计算机的设备 `/dev/input/mice`。但是一般情况下，该设备只有超级用户才具有访问权限。解决的方法有两类：或者让程序具有超级用

户的有效权限；或者改变设备的既有访问权限，以便让普通用户也可以访问。

后一种方法相对简单，并且易于理解。使用一条命令就可以解决访问权限的问题：

```
sudo chmod +r /dev/input/mice
```

想要以更加专业地方式解决设备文件访问权限问题，需要研究并修改 udev 的规则配置。通过修改 /etc/udev/udev.conf 文件，或者在 /etc/udev/rules.d/ 目录下增加专门的规则定制文件。详情请查找 udev 有关的资料，

计算机 A 和计算机 B，在实验室中，通过相邻的两台计算机模拟。使用 ifconfig 命令了解计算机的 IP 地址。

在程序的调试阶段，也可以在同一台计算机上，运行分别运行两个程序，以模拟计算机 A 和计算机 B。这种情况下，使用自环地址 127.0.0.1，作为 IP 地址。

3. 实验环境测试

有些时候，你所编写的程序不能正常运行，并非是程序自身的问题，也有可能是硬件、网络或者操作系统的原因。也有可能是你本身的测试方法不正确所导致。

快速地将外部原因与自己所编写程序导致的问题区分开来，对于调试程序的过程来说是非常重要的。可以使用 Linux 系统自身包含的命令与工具，快速地测试硬件、网络以及系统是否正常。

先在计算机 B 上运行命令：

```
nc -l 9012 | xxd
```

其中的 9012 是随便选取的一个空闲 IP 端口。

然后在计算机 A 上运行：

```
sudo cat /dev/input/mice | nc 192.168.4.123 9012
```

其中 192.168.4.123 是计算机 B 的 IP 地址，实验时使用实际的 IP 地址。

如果一切正常，在计算机 A 上移动鼠标，从计算机 B 的终端窗口内，就可以看到发送过来的数据。小心地在桌面上水平或者竖直移动鼠标，观察数据的变化规律。

4. 示范程序

为了给设计提供一些思路，下面给出一种简单的设计和代码作为参考。

程序 serv.c 作为服务端，运行在计算机 B 上，创建一个套接字，并且在端口 8765 上监听连接请求。当有连接请求时，接受它并且创建一个线程负责接收客户端发送过来的信息。

程序 cli.c 作为客户端，运行在计算机 A 上，读取鼠标数据，并使用一个套接字，将这些数据发送到服务端。

在两台计算机上分别编译并且运行它们，就可以看到实际的效果。

在计算机 A 上，编译程序：

```
make serv CFLAGS=-pthread
```

运行：

```
./serv
```

在计算机 B 上，编译程序：

```
make cli
```

修改鼠标设备文件的访问权限：

```
sudo chmod +r /dev/input/mice
```

运行程序:

```
./cli <SERVER_IP_ADDR>
```

注意上面的<SERVER_IP_ADDR>是指服务端的 IP 地址, 例如: 192.168.5.32

程序 serv.c 如下:

```
#include<stdio.h>
#include<stdio.h>
#include<string.h> //strlen
#include<stdlib.h> //strlen
#include<sys/socket.h>
#include<arpa/inet.h> //inet_addr
#include<unistd.h> //write

#include<pthread.h> //for threading , link with lpthread

void *connection_handler(void *);

int main(int argc , char *argv[])
{
    int socket_desc , new_socket , c , *new_sock;
    struct sockaddr_in server , client;
    char *message;

    //Create socket
    socket_desc = socket(AF_INET , SOCK_STREAM , 0);
    if (socket_desc == -1)
    {
        printf("Could not create socket");
    }

    //Prepare the sockaddr_in structure
    server.sin_family = AF_INET;
    server.sin_addr.s_addr = INADDR_ANY;
    server.sin_port = htons( 8765 );

    //Bind
    if( bind(socket_desc,(struct sockaddr *)&server , sizeof(server))
< 0)
    {
        puts("bind failed");
        return 1;
    }
    puts("bind done");

    //Listen
    listen(socket_desc , 3);

    //Accept and incoming connection
    puts("Waiting for incoming connections...");
    c = sizeof(struct sockaddr_in);
    while( (new_socket = accept(socket_desc, (struct sockaddr
*)&client, (socklen_t*)&c)) )
    {
        puts("Connection accepted");

        //Reply to the client
        message = "Hello Client , I have received your connection. And
now I will assign a handler for you\n";
        write(new_socket , message , strlen(message));
```

```

        pthread_t sniffer_thread;
        new_sock = malloc(1);
        *new_sock = new_socket;

        if( pthread_create( &sniffer_thread , NULL ,
connection_handler , (void*) new_sock) < 0)
        {
            perror("could not create thread");
            return 1;
        }

        //Now join the thread , so that we dont terminate before the
thread
        //pthread_join( sniffer_thread , NULL);
        puts("Handler assigned");
    }

    if (new_socket<0)
    {
        perror("accept failed");
        return 1;
    }

    return 0;
}

/*
 * This will handle connection for each client
 * */
void *connection_handler(void *socket_desc)
{
    //Get the socket descriptor
    int sock = *(int*)socket_desc;
    int read_size;
    char *message , client_message[2000];

    //Send some messages to the client
    message = "Greetings! I am your connection handler\n";
    write(sock , message , strlen(message));

    message = "Now type something and i shall repeat what you type \n";
    write(sock , message , strlen(message));

    //Receive a message from client
    while( (read_size = recv(sock , client_message , 2000 , 0)) > 0 )
    {
        //Send the message back to client
        //write(sock , client_message , strlen(client_message));
        puts(client_message);
    }

    if(read_size == 0)
    {
        puts("Client disconnected");
        fflush(stdout);
    }
    else if(read_size == -1)
    {
        perror("recv failed");
    }
}

```

```

    //Free the socket pointer
    free(socket_desc);

    return 0;
}

```

程序 cli.c 如下:

```

#include<stdio.h>
#include<string.h>    //strlen
#include <fcntl.h>
#include<sys/socket.h>
#include<arpa/inet.h> //inet_addr

int main(int argc , char *argv[])
{
    int socket_desc, fd;
    struct sockaddr_in server;
    char message[1024];

    //Create socket
    socket_desc = socket(AF_INET , SOCK_STREAM , 0);
    if (socket_desc == -1)
    {
        printf("Could not create socket");
    }

    server.sin_addr.s_addr = inet_addr(argv[1]);
    server.sin_family = AF_INET;
    server.sin_port = htons( 8765 );

    //Connect to remote server
    if (connect(socket_desc , (struct sockaddr *)&server ,
sizeof(server)) < 0)
    {
        puts("connect error");
        return 1;
    }

    puts("Connected\n");

    //Read data from mice and send them to server from socket
    if ( (fd = open("/dev/input/mice", O_RDONLY)) == -1 )
    {
        perror("cannot open /dev/input/mice");
        return 1;
    }
    while ( read(fd, message, 1024) > 0 )
    {
        if( send(socket_desc , message , strlen(message) , 0) < 0)
        {
            puts("Send failed");
            return 1;
        }
    }

    return 0;
}

```

需要说明的是，这里给出的程序，仅仅是能够运行并且实现了要求的功能，但是仍然存在许多问题，需要进一步完善，才能有比较好的演示效果。这些改进工作留给同学在实验中完成。

5. 扩展设计

除了基本的要求之外，有兴趣的同学可以对所设计的程序进行扩展。比如添加 GUI 界面；从两台计算机之间的传送温度数据，扩展至多台计算机之间构成更加复杂的温度测量网络。

附录 A Linux 系统应用程序编译

1. 建立程序。

在用户主目录下，创建一个名为 prog 的子目录。

转到该目录下，使用 vim 编辑器，或者任何其他你所喜欢的编辑器，创建一个文件名为：first.c 的文件。其内容可以是一段最为简单的 C 程序。附录中提供了一段 first.c 的示范代码可供参考。

2. 编译和运行

使用以下命令编译、链接，并生成可执行的程序：

```
gcc first.c
```

在没有特别指明的情况下，所生成的可执行文件，总是 a.out。

执行程序的方法是，在命令行上输入：

```
./a.out
```

注意：必须在可执行的文件名之前，显式地指明路径，绝对路径和相对路径均可。除非在指示搜索路径的 shell 环境变量 \$PATH 内，指明了用户程序的路径，即便是可执行文件就在当前目录，也需要在文件名之前加上“./”这两个字符。

如果编译、链接的过程中出现错误，将不会生成可执行文件。请按编译器输出的错误信息，检查源程序并加以改正。

实际工作中，很少有人愿意接受 a.out 这样意义含混的文件名，作为可执行程序的文件名。通常会在编译选项中指定更加便于识别名字。一般的做法是使用去掉后缀的文件名。

```
gcc first.c -o first
```

以上编译命令所生成的文件，就是没有后缀名的可执行程序 first。

执行它的命令是：

```
./first
```

3. 分步编译、链接

选项 -c 明确指示编译器仅仅编译源文件，仅仅生成编译后的目标文件，并不生成最终的可执行程序。

```
gcc -c first.c
```

将会生成目标代码 first.o。也可以明确地指出输出目标文件的名字：

```
gcc -c first.c -o first.o
```

单独的链接过程如下：

```
gcc first.o -o first
```

当存在多个目标模块时，链接命令可能类似于：

```
gcc main.o module1.o module2.o module3.o -o myprog
```

这当中的 main.o, module1.o, module2.o, module3.o 分别是源文件 main.c, module1.c, module2.c, module3.c 经过单独编译得到。

有时需要更加详尽地了解编译的过程，使用 -E 选项可以得到预处理之后的程序。

```
gcc -E first.c -o first.i
```

使用 -s 选项，编译器将会生成汇编语言，而非机器语言的代码：

```
gcc -S first.c -o first.s
```

如果一切正确，用编辑器打开 first.s，可以看到编译生成的汇编语言程序。

参考代码：

```
first.c

/* Program that shows how fork system call works */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main(void)
{
    pid_t pid;
    char cwdir_cld[100];
    char cwdir_prt[100];

    if ((pid = fork()) < 0) {
        perror("fork error");
        exit(1);
    }

    if (pid == 0) {
        /*
         * Now we are in the childs thread
         * let us issue a chdir syscall :)
         */
        if(chdir("/tmp") < 0) {
            perror("chdir error");
        }

        /*
         * Let us now print the current working directory of the child
         */
        if (getcwd(cwdir_cld, 100) != NULL) {
            printf("Current working directory of the child is : %s\n",
cwdir_cld);
        } else {
            perror("getcwd error");
        }

        /* end of the child */
    } else {
        /*
         * Now we are in the parent thread
         * Let is check the current working directory
         */
        if (getcwd(cwdir_prt, 100) != NULL) {
            printf("Current working directory of the parent is :
%s\n", cwdir_prt);
        } else {
            perror("getcwd error");
        }
    }
}
```



```
    }  
    printf("\n");  
    return(0);  
}
```

附录 B 程序的调试

1. 经典的程序调试的手段

除了及其简单的程序，绝大多数程序很难不经调试，简单地编译、执行，就发现它完全实现了预定功能。排查程序中存在的错误是程序员的重要工作。

没有经验的初学者，往往通过对着屏幕反复阅读程序的办法，试图发现程序的错误。但是这往往是徒劳的。

程序的错误可以分为两大类，第一类是编译和链接时的错误。编译器会报告这一类错误，并且指明错误所在的行，通过检查出错部分的前后语句，通常能够很快地发现问题。排除这类错误并不困难。程序的调试技术，主要是用来应对程序的第二类错误：运行时的错误。或者程序可以运行，但是所期待的结果不符。排除这类错误的过程称为 debug，而 bug 就是指程序中的一个缺陷。

程序在计算机上执行时，已经被编译成了机器语言，在机器语言的级别上排错更加困难，因为源程序和机器语言之间的对应关系不容易确定。检查这一类问题，往往要花费程序员大量的时间和精力。

最直截了当的想法是，先要定位出错的位置。这种想法是正确的。有一种经典的方法，不需要任何额外的工具，就可以使用。这一方法因为简单易行，至今仍被许多程序员使用：在程序的关键部位插入 `printf` 语句，将重要的信息输出到屏幕上显示出来，或者记录到一个临时文件中，通过检查这些输出的信息推断错误的地方。

这一方法的缺点是，一方面所增加的语句可能本身会引入问题，形成干扰；另一方面，当调试结束后，需要逐行删去加入的语句。这一过程，也可能引入新的错误。

2. 程序调试工具

Linux 系统上，著名的程序调试工具当属 GNU gdb。gdb 可以在源代码的级别上调试程序，功能强大。实际上，它不仅仅可以用来调试 C 程序，但 C 是它最擅长的方面。

gdb 可以在下述方面对我们调试程序的工作有所帮助：

- 1) 运行程序
- 2) 设置断点
- 3) 单步执行
- 4) 显示源码
- 5) 编辑源码
- 6) 访问和修改变量
- 7) 跟踪程序执行
- 8) 检查堆栈活动
- 9) 搜索函数、变量、目标
- 10) 显示变量和表达式的值

gdb 也有基于图形界面的版本，但从文本方式的 gdb 入手是更基本的。gdb 所做的工作归纳起来为 4 件事：

- 1) 设置运行环境和参数，运行程序

- 2) 让程序在指定条件下停止
- 3) 当程序停止时, 检查发生了什么
- 4) 改变正在调试的程序以帮助排错

简单地说, gdb 帮助我们检查、了解、控制、程序的执行过程。

3. 使用 gdb 之前

gdb 并不是直接调试源程序文件本身, 而是针对编译之后的可执行文件进行。

为了使 gdb 能够很好地工作, 需要在可执行的程序文件中包含一些用于调试的额外信息。这需要在编译时, 加上指定的参数, 例如:

```
gcc -g myprg.c -o myprg
```

其中 -g 选项指示编译器为可执行文件加入调试用的信息。这样所得到的文件比原先略大一些。注意, 打算调试的程序, 不要使用优化编译选项。优化让机器码和源程序的语句失去严格的对应关系, 使程序的调试更加困难。

4. gdb 的启动与退出

假如你有一个程序已经按上面的方法编译完毕, 可以按以下方式启动 gdb:

```
$gdb mypry
```

一串版权信息之后, 出现

```
(gdb)
```

这就是 gdb 的提示符, gdb 本身的种种命令, 可在此提示符后输入。

退出 gdb 的方法是输入 quit 命令

或者按 Ctrl-D 组合键

Ctrl-C 不会中断 gdb 的运行, 而是中断被调试程序的运行。

5. 基本的 gdb 命令

帮助

help 随时获得帮助, 获得命令列表

Help 后跟单个命令, 列出命令更详尽的资料

查看代码

list 列程序代码, 缺省为 10 行, 再执行一次, 又 10 行

list 4, 9 列 4 至 9 行

list 函数名

list 命令之后, 空行直接回车, 相当于又一个 list 命令

运行程序

run 执行程序, Ctrl-C 中断, run 继续或重新开始

设置断点

break 行号

break 函数名

break 行号或函数名 if 条件 例如: break 9 if result > 50

watch 条件 不指定特定的断点位置, 只要 result 值符合条件, 立刻中断

(注意, 只有运行过变量声明之后, 才可使用此种方式)

例如: `watch result > 50,`

观察变量

`print` 变量名 查看变量的值

`whatis` 变量名 查看变量类型

执行控制

`continue` 程序在中断之后继续运行（命令允许简写，并使用 TAB 键自动补全）

`step` 执行一行。如果下一行是一个函数调用，`step` 将会进入函数，执行函数内的第一行操作。

`next` 与 `step` 相似，但将函数调用看作一条语句，一次将整个函数执行完毕。

管理程序中的中断点

`info breakpoints` 查看已设置的断点信息

输出的信息如下：

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep	y	0x8048490	in main at myprg:6
编号	断点类型	保持	生效	当前地址	位置

`disable` 断点编号 使指定断点失效

`disable` 不带参数 使所有断点失效

`enable` 使断点生效

`enable once` 使断点一次性生效

`delete` 删除断点

`clear` 行号（而不是接断点编号）或函数名 删除断点

`clear` 不带参数 删除程序中全部中断点

附录 C 项目管理与自动化工具

1. 基础知识

假设你的程序，仅仅由单一的源代码文件构成，编译和链接的过程将会非常简单。但是，在真实的工程环境之下，一个项目往往由许多源文件构成。这些文件甚至可能分布在多级子目录中，包括头文件、配置文件、数据文件、资源文件等等。

包含多个模块的项目，在编译时，各模块往往先各自被编译成一个个目标文件，最后再将这些目标文件、库文件、系统缺省库等等，一起连接成一个可执行的文件。

例如，你的程序包含以下的文件：

```
main.c  part1.c  part2.c  part3.c
```

你需要这样编译：

```
gcc -c main.c
```

```
gcc -c part1.c
```

```
gcc -c part2.c
```

```
gcc -c part3.c
```

编译选项 `-c` 告诉 `gcc` 仅编译，不进行链接。这样就会分别生成 4 个名字以 `.o` 为结尾的目标文件。

再执行：

```
gcc -o myprg main.o part1.o part2.o part3.o
```

链接这些目标文件，生成可执行文件。

当然，以上的步骤也可以合并在一起，让编译和链接在一个命令中自动完成：

```
gcc -o myprg main.c part1.c part2.c part3.c
```

这看似省事，然而一旦修改其中的任一个文件，都要对所有文件全部编译一遍，才能获得可执行文件。当文件很多时，这是很费时的。大型的程序，将所有的模块完整地编译一遍，可能需要耗费数个小时，甚至更长。

更加合理的方式是，只重新编译那些与改动有关的部分文件。没有改动的模块，它们的目标文件不需要重新生成，仍然可以在链接时使用。只不过，当项目中包含的文件繁多时，每修改一个文件后，究竟应该重新编译哪些模块，哪些模块的目标文件无需重新编译？判断这些破费脑筋。如何将这一切自动地进行呢？这就是 `make` 的任务。

`make` 是一个实用程序。它读取一个特定的文件，该文件的名字通常是 `Makefile` 或者 `makefile`，然后按照文件当中的指示去自动工作。若文件中没有指明的规则，或者干脆不存在 `Makefile` 或者 `makefile` 文件时，`make` 程序凭借着其内置的缺省规则，尽可能地完成任务。

2. 运行 make

启动 make 程序最简单的方式，不带任何选项或参数，直接在命令行输入：

```
make
```

并回车即可。

make 命令的完整的形式为：

```
make [选项] [目标] [宏定义]
```

make 后面的[选项]，[目标]，[宏定义]可以按任何顺序。

最常用的用法是：

make [目标]，如：

```
make clean
make install
make all
```

其中的目标，应该是在 Makefile 中定义了的。

3. Makefile 的编写

Makefile 是一个描述性的文件，它描述了三个内容：

- 1) 需要建立的目标有哪些；
- 2) 建立这些目标需要哪些文件作为前提，也就是依赖关系；
- 3) 建立这些目标文件所要执行的命令。

一个 Makefile 范例文件的内容如下：

```
# This is a sample of Makefile
# Written by David for learning
myprg: main.o part1.o part2.o part3.o
    gcc -o myprg main.o part1.o part2.o part3.o
main.o: main.c
    gcc -c main.c
part1.o: part1.c head.h
    gcc -c part1.c
part2.o: part2.c head.h
    gcc -c part2.c
part3.o: part3.c head.h
    gcc -c part3.c
```

特别注意，含有命令的一行，必须用水平制表符（也称“跳格键”，Tab）开始，使用多个空格代替是不正确的。

观察以上文件有以下要点：

用#开头的行是注释

文件描述了依赖关系

myprg	依赖	main.o, part1.o, part2.o 和 part3.o
main.o	依赖	main.c
part1.o	依赖	part1.c 和 head.h
part2.o	依赖	part2.c 和 head.h
part3.o	依赖	part3.c 和 head.h

（思考：倘若修改了 head.h 文件，哪些部分是需要重新编译的？）

依赖关系由五个项目 (Entry) 定义。格式为最左端的是目标跟冒号, 后面是用空格隔开的所依赖目标的列表。

文件描述了每个目标的产生方法: 在项目的下一行, 用 TAB 字符开头, 给出了产生目标所需要执行的命令。

4. 其他的生成项目

除了编译和链接操作以外, 还可以在 Makefile 内增加其他项目:

我们在前述 Makefile 文件内增加:

```
clean:
    rm -f *.o myprg
install:
    cp -f myprg ~/usr/local/bin
```

这样之后, 将来通过 make clean 命令就将工作目录下的编译结果和其他中间文件一并删除, 以便保持源代码目录内的整洁。

使用 make install 命令, 则将最终的可执行程序, 安装到指定目录下。

直接键入不带目标参数的 make, 实际上是对第一个目标进行操作, 在所给的范例当中, 也就是相当于执行 make myprg。

5. Makefile 的简化和宏的使用

当工程中包含的文件数比较多时, Makefile 当中的行可能变得很长, 这时, 可使用 make 内部定义的变量来简化规则的表达:

- \$* 代表目标文件的名字去掉后缀部分
- \$@ 代表目标文件的全名
- \$< 该目标的第一个相关文件名
- ^ 代表规则的所有依赖文件列表, 使用空格分隔

如:

```
part1.o: part1.c head.h
    gcc -c $*.c
```

其中的 \$* 代指 part1

又如:

```
part2.o: part2.c head.h
    gcc -c -o $@ $<
```

其中的第 2 行相当于:

```
gcc -c -o part2.o part2.c
```

利用后缀规则简化 Makefile 的方法:

```
.c.o:
    gcc -c $<
```

上面两行在 Makefile 文件中, 指明后缀为 .c 的文件是源文件, 用来生成后缀为 .o 的目标文件, 生成的方法是使用 gcc 带 -c 选项编译。

这如果用模式规则写出来, 就是:

```
%o: %c
gcc -c $<
```

这样的两行具有同样的意义。

make 允许用户在 Makefile 文件内定义自己的变量,从而使一些字符串具有特殊含义。如:

```
CC := gcc
OBJS := part1.o part2.o part3.o
${CC} -o myprg ${OBJS}
```

第三行的效果就相当于:

```
gcc -o myprg part1.o part2.o part3.o
```

6. 关于 make 的深入学习

make 还有一些更为复杂的规则 可参见有关手册。

当程序在不同场合下编译时,由于编译环境的不同,所需的 Makefile 文件有一些微妙的差异。编写在各种场合下都能正常工作的 Makefile 也可能成为一种困难的工作。为此,手工一行一行地去写 Makefile 文件成为一件费时费力又容易出错的事情。人们希望可以由某种自动化的工具来,根据一定的条件,自动判断系统的环境,自动地生成 Makefile 文件。完成这样的工作,需要更多的工具: automake 和 autoconf 都是为了实现这一目的而存在的。为了让 automake 和 autoconf 工作,又需要 m4 宏解释器及 perl 的支持。这就形成 Linux 环境之下编程工具链的复杂性。

按这种方式工作的项目,在编译之前,首先运行

```
./configure      产生 Makefile, 然后
```

```
make             编译
```

```
make install    安装
```

```
make clean      清除编译过程中产生的文件 (为另一次全新的编译过程准备)
```


附录 D 版本控制与源代码管理

1. 版本控制与源代码管理基础知识

在真实的开发实践中，程序源代码以及重要的数据、资源文件、配置文件等等，都是公司的重要资产。这些文件的损坏或丢失，是无法承受的灾难，必须可靠地存放。而开发过程中产生的各种版本，极易造成管理上的负担。使用专门的工具处理这些问题，就变得十分必要。在许多版本控制工具之中，包括 Linux 内核在内的许多重要项目，都使用 Git 作为版本管理工具。

2. 检查本机上是否安装 Git 程序

在命令行执行命令：

```
git --version
```

观察输出结果。如果显示出 git 自身的版本信息，表明 git 已安装在本机。如下：

```
git version 1.7.9.5
```

所显示的具体版本号，可能会有差异。

3. 创建一个项目的子目录，并在子目录增加源程序文件

可以复制其他实验的代码，用在此处作为实验性代码。

创建本地 git 仓库：

```
git init
```

观察 git 仓库的实际位置：

```
ls -a
```

```
ls -R .git
```

4. 添加项目中源代码文件到 git 仓库

```
git add <文件名>
```

5. 向 git 仓库中提交内容

```
git commit
```

这个命令会自动调用缺省的文本编辑器，要求输入此次提交的备注信息。应该在编辑器从输入有意义，具有说明性的文字，然后保存并退出编辑器。提交命令会自动接着执行。你也可以使用命令：

```
git commit -m '<关于本次提交的说明信息>（中英文均可）'
```

通过选项 -m 直接在命令行中说明信息。说明信息应该用单引号或者双引号括住。其中的内容为你打算做出的说明。

6. 观察仓库的状态

执行 git status 命令，观察输出结果。

修改你的程序之后，再次执行 git status 命令，观察输出结果。

然后, 执行 `git add <刚才修改过的文件>`, 再执行 `git status` 命令, 观察输出结果。

执行 `git commit -m '<注释>'` 命令之后, 再执行 `git status` 命令, 观察输出结果。

7. 创建分支

```
git branch dev
```

8. 切换到新的分支

```
git checkout dev
```

9. 在新的分支中修改代码并提交

按前述方法, 有意识地修改代码。注意记录关键的修改之处。以便在后续的实验中进行比对。

注意不要忘记向 `git` 仓库提交所做的修改。

10. 切换到主分支

使用命令:

```
git checkout master
```

将工作目录切换回到主分支。注意, 通常将 `master` 分支作为代码的主分支。这是一般的惯例, 并非强制的要求。

11. 查看分支之间的独立性

在 `master` 分支中, 观察代码的改动情况。体会 `master` 分支和 `dev` 分支之间的独立性。可以分别使用 `git checkout dev` 和 `git checkout master` 反复在分支之间切换, 体会分支的含义。

尝试以下命令:

```
git branch
```

显示当前所在分支

```
git show-branch
```

了解分支情况

```
git log
```

显示分支的修改历史

```
git log --graph --all --pretty=oneline
```

绘出分支历史关系图

12. 分支间的合并

在 `master` 分支中, 将 `dev` 分支中所做的更新合并过来:

```
git merge dev
```

观察代码的改动情况。体会合并的意义。

观察分支历史关系图, 注意其中的合并线索。

13. 自主学习尝试其他 `git` 命令

尝试观察源代码的历史版本。

设置远程仓库, 并联系远程仓库与本地仓库之间的同步关系。

在 `coding.net` 上创建账号并将自己的仓库同步到 `coding.net`。

从 `coding.net` 上使用 `git clone` 命令获得已有项目的 `git` 仓库。

重要提示：

请将你在 coding.net 上的账户名和密码妥善保管。

在本课程学习期间，我们鼓励你持续地向你的代码仓库提交你的代码。你在代码仓库中，针对项目中文档、代码的更新，整个项目的开发分支及其变动历史，对于我们评价你的学习有重要影响。

你需要在期末考核阶段，提供自己的 coding.net 个性后缀名（Global Key）、项目名、访问你项目仓库的网络地址（URL），但不会有人向你索取密码，你应当保持账户密码不被他人所知。

coding.net 或者类似的项目托管网站，允许你选择仓库的性质为公开或私有，这种情况下，请一定选择公开，以便让我们可以访问你的仓库。

如果你的 coding.net 账户下，存在多个项目，请注明哪一个项目是用于本课程考核的。如果不特别说明，那么所有的项目，都可能被列入考查范围。